SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. *Introduction:*

1.1. *Purpose:*

Inventra is an AI-powered operations intelligence system designed for F&B kiosk owners in Malaysian shopping malls. It provides real-time analytics, demand forecasting, and an AI-driven business expansion planning system.

The purpose of this System Analysis Document is to define the **technical architecture, system workflows, and design decisions** behind Inventra.

The key elements covered are:

1. ***Architecture***
   - Inventra is a cloud-based, full-stack system that combines a modern web application (Next.js) with a distributed backend powered by multiple AI agents.
   - The backend follows a hybrid architecture using **MCP (Model Context Protocol)** for tool-based operations (SQL and RAG) and **A2A (Agent-to-Agent)** communication for complex multi-step reasoning tasks.
   - All components are designed to be modular, scalable, and independently deployable.
2. ***Data Flows***
   - User interactions originate from the frontend dashboard or AI chat interface and are routed through the CopilotKit API layer.
   - Requests are processed by a central orchestrator agent, which dynamically decides whether to query structured data (Supabase), retrieve contextual knowledge (RAG), or delegate tasks to specialized AI agents.
   - The processed data is then returned to the frontend as real-time insights, visual analytics, or structured decision outputs.
3. ***Model Process***
   - The system implements two main AI workflows:
     i. A **multi-agent expansion planning pipeline**, where five specialized agents collaboratively generate business strategies from location selection to final roadmap.
     ii. An **ML forecasting pipeline**, which predicts demand, revenue, stockout risk, and inventory needs using historical data.
   - These workflows demonstrate end-to-end intelligent decision support rather than simple data retrieval.
4. ***Role of Reference***
   - This document serves as a reference for developers, designers, and evaluators to understand the overall system architecture, data handling mechanisms, and AI integration strategy.
   - It also ensures clarity in implementation, testing, and future scalability of the system.

© UMHackathon 2026

1.2. Background:

The Food & Beverage (F&B) kiosk industry in Malaysian shopping malls is highly competitive and operationally complex. Kiosk owners must continuously manage fluctuating customer demand, limited inventory space, and tight profit margins while making strategic decisions such as pricing, staffing, and business expansion.

However, most kiosk operators still rely on manual decision-making or basic point-of-sale (POS) systems, which only provide historical data without actionable insights. As a result, several key challenges arise:

- **Inaccurate demand forecasting**, leading to stock shortages during peak hours or excess inventory and food waste during slow periods
- **Lack of real-time operational visibility** across multiple outlets
- **Inefficient inventory management**, especially for perishable goods with expiry constraints
- **High-risk business expansion decisions**, often based on intuition rather than data

These limitations create a strong need for an intelligent system that not only tracks operations but also supports predictive and strategic decision-making.

1. Previous Version

Existing solutions in the market primarily include:

- **POS systems** that track transactions and sales history
- **Basic inventory systems** that monitor stock levels
- **Standalone analytics dashboards** with limited forecasting capability

While these systems provide useful data, they are:

- **Fragmented** (not integrated into one intelligent system)
- **Reactive rather than proactive** (focus on past data only)
- **Lacking AI-driven insights** for forecasting and expansion

There is no unified platform that combines real-time analytics, predictive intelligence, and strategic planning in one system.

2. Changes in Major Architectural Components

Inventra introduces several major architectural improvements compared to traditional systems:

- Transition from **single-system dashboards** → **multi-agent AI architecture**
- Integration of **LLM (Gemini) as a decision engine**, not just a chatbot
- Use of **MCP (Model Context Protocol)** for structured data retrieval (SQL + RAG)
- Implementation of **A2A (Agent-to-Agent) communication** for complex workflows

© UMHackathon 2026

- Adoption of a **cloud-based, modular microservices design** enabling scalability

This shift allows the system to move beyond simple data display into intelligent, automated decision support.

3. New Capabilities Introduced

Inventra introduces several new capabilities that are not available in traditional systems:

- **AI-Powered Business Expansion Planning**
  - Multi-agent pipeline that evaluates location, financial feasibility, market strategy, and risk before generating a strategic roadmap
- **Machine Learning Demand Forecasting**
  - Predicts future orders, revenue, stockout risk, and inventory needs
- **Natural Language Business Intelligence**
  - Users can query business data using plain English via AI chat
- **Real-Time Multi-Outlet Analytics**
  - Centralized dashboard for monitoring performance across multiple locations
- **3D Inventory Visualization**
  - Interactive interface for tracking stock levels and expiry status in real time

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Kiosk Owners (Primary Users)	Monitor daily operations, analyze performance, manage inventory, and make business expansion decisions using the system	Accurate real-time analytics, AI-driven recommendations, easy-to-use dashboard, and reliable decision support for expansion
Outlet Managers / Staff	Manage stock levels, update inventory, and monitor daily kiosk operations	Real-time stock updates, clear alerts for low stock and expiry, simple and efficient inventory management interface
Business Analysts / Decision Makers	Analyze trends, evaluate performance across multiple outlets, and plan strategic growth	Comprehensive insights, forecasting accuracy, and clear visualization of business performance and risks

© UMHackathon 2026

AI System (Multi-Agent Architecture)	Process user queries, perform data analysis, generate forecasts, and execute multi-step decision workflows	High accuracy, fast response time, structured outputs, and efficient coordination between agents
Development Team	Design, develop, and maintain the platform (frontend, backend, AI agents)	Modular architecture, clear API communication, scalable system design, and maintainable codebase
QA Team	Test system functionality, validate AI outputs, and ensure system reliability	Well-defined requirements, consistent data flow, edge case handling, and stable system performance
External API Providers (Google Maps, Foursquare, ML Services)	Provide data for location analysis, market research, and forecasting	Reliable API integration, minimal latency, and consistent data availability

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview:

Type	**Details**
System	Web-Based AI Operations Intelligence Platform (Next.js Dashboard + AI Chat Interface)
Architecture	Cloud-based, Multi-Agent Microservices Architecture with LLM Integration

© UMHackathon 2026

Email: umhackathon@um.edu.my

Inventra is structured as a client-server-based system that integrates a modern web frontend with a distributed AI backend composed of multiple specialized agents.

The system consists of three main layers:

1. Frontend Layer (Client)

- Built using **Next.js and React**
- Provides:
  - Real-time analytics dashboard
  - 3D inventory management interface
  - Business planning canvas
  - AI chat interface (CopilotKit)
- Acts as the primary interaction point for kiosk owners and managers

2. Backend Orchestration Layer

- A central **Orchestrator Agent (Gemini 2.5 Flash)** handles all incoming user requests
- Responsible for:
  - Understanding user intent
  - Routing tasks to the correct component:
    - SQL queries (structured data)
    - RAG queries (contextual insights)
    - A2A agents (complex workflows)
- Uses:
  - **AG-UI Protocol** for communication with frontend
  - **MCP (Model Context Protocol)** for tool-based operations
  - **A2A (Agent-to-Agent)** for delegation to specialized agents

3. Data & AI Service Layer

This layer contains all processing and intelligence components:

a. SQL MCP Server

- Handles structured data queries
- Connects directly to **Supabase PostgreSQL database**
- Used for:
  - Revenue, orders, inventory, expiry data

b. RAG MCP Server

- Performs semantic search using embeddings
- Uses:

© UMHackathon 2026

  - OpenAI embedding model
  - Vector database (Supabase pgvector)

c. A2A Specialized Agents

- Independent AI agents for complex reasoning tasks:
  - Site Selection Agent
  - Expansion Feasibility Agent
  - Market Research Agent
  - Risk Manager Agent
  - Strategic Planner Agent
  - Forecast Agent
- Each agent:
  - Runs independently
  - Communicates via HTTP
  - Returns structured JSON outputs

4. External Services Integration

Inventra integrates multiple external APIs:

- **Google Maps API** → Location & competitor analysis
- **Foursquare API** → Foot traffic estimation
- **Bank Negara API** → Financial indicators (OPR rate)
- **Malaysia Open Data API** → Demographics
- **Cloud Run ML Service** → Forecast predictions

5. Database Layer

- **Supabase (PostgreSQL)** as primary database
- Stores:
  - Orders
  - Inventory
  - Expiry data
  - Location data
- Includes **vector database** for RAG-based search

2.1.2 LLM as Service Layer

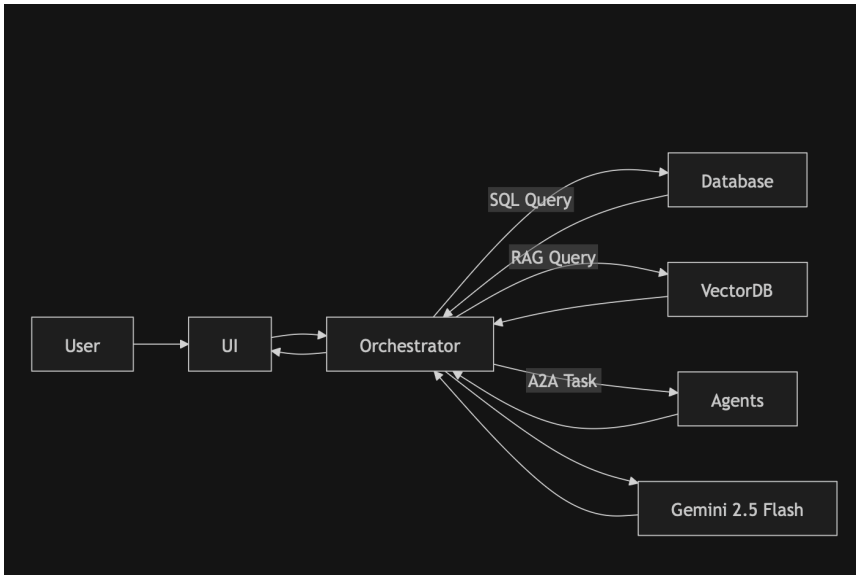
Unlike traditional systems where AI is treated as a standalone module, Inventra integrates the LLM (**Gemini 2.5 Flash**) as a **core service layer** that drives system intelligence and decision-making.

© UMHackathon 2026

How LLM is Used in the System

1. **Prompt Construction**
   - User input is combined with:
     - Structured data (SQL results)
     - Contextual knowledge (RAG results)
     - System instructions (agent-specific tasks)
2. **Context Window**
   - Includes:
     - User query
     - Relevant database outputs
     - Top-k semantic search results (RAG)
   - Limited to ensure efficiency and reduce token cost
3. **Response Processing**
   - LLM outputs structured JSON responses
   - Parsed by orchestrator
   - Passed to:
     - Frontend UI (charts, cards)
     - Next agent in pipeline
4. **Token Optimization**
   - Context truncation applied
   - Only top relevant data is passed
   - Repeated queries cached

System Interaction Flow (LLM-Centric)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.1.3 Dependency Diagram

The dependency diagram illustrates how the Large Language Model (LLM) is integrated as a core service layer within Inventra, showing the interaction between frontend, backend, AI agents, database, and external APIs.

a. Prompt Construction

Prompts are constructed at the Orchestrator Agent layer by combining:

- User query (from frontend)
- Structured data (SQL MCP results)
- Contextual data (RAG MCP results)
- Task-specific instructions (based on agent type)

This ensures the LLM receives rich, contextual input instead of raw user text.

b. Context Window Composition

The LLM context window includes:

- User intent
- Relevant database outputs (e.g. revenue, stock)
- Top-k retrieved documents from RAG (semantic search)
- Agent-specific instructions

To optimize performance:

- Only top 5–6 relevant results are included
- Irrelevant or large data is filtered out

c. Response Parsing & Flow

- The LLM returns structured JSON outputs
- The Orchestrator:
    - Parses the response
    - Validates structure
    - Routes it to:
        - Frontend (for visualization)
        - Next A2A agent (for pipeline continuation)

© UMHackathon 2026

d. Token Limitation & Chunking

Token control is enforced at the Orchestrator:

- Large inputs are chunked before sending to LLM
- Only summarized or relevant data is passed
- Repeated queries are cached to reduce token usage

e. Major API Calls Between Components

Key interactions include:

- Frontend → `/api/copilotkit` (user queries)
- CopilotKit → Orchestrator (AG-UI protocol)
- Orchestrator → SQL MCP (`/sse`)
- Orchestrator → RAG MCP (`/sse`)
- Orchestrator → A2A Agents (`http://localhost:902x`)
- A2A Agents → External APIs (Google Maps, Foursquare, ML Service)

f. System Interaction Across Components

- Frontend handles user interaction and visualization
- Backend orchestrator controls logic and routing
- Database stores structured business data
- External APIs provide real-world data
- AI agents perform reasoning and predictions

All components are **loosely coupled and modular**.

g. Distributed System Design

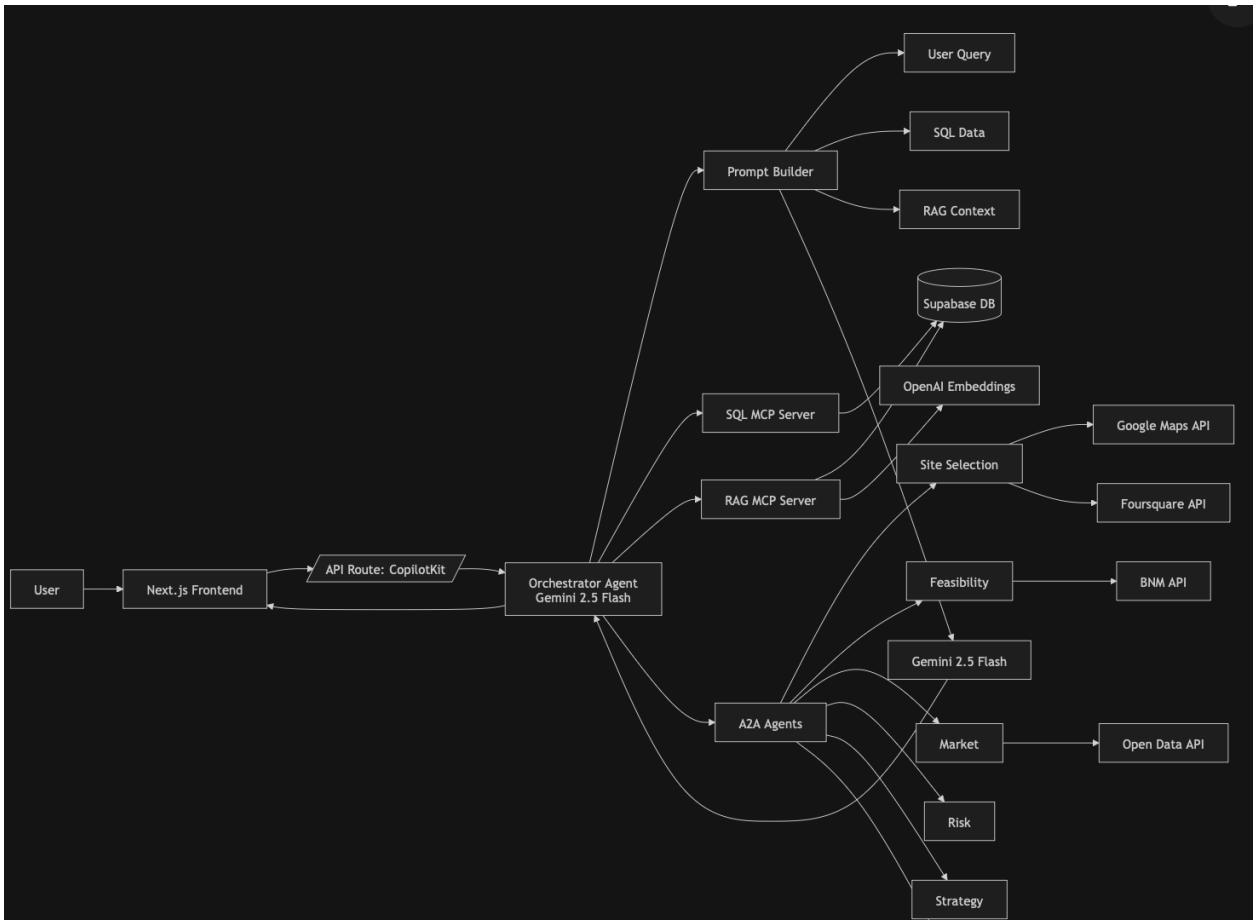
Inventra is designed as a **distributed system**:

- Each AI agent runs independently on separate ports
- MCP servers operate as independent services
- External APIs are integrated as separate dependencies
- This allows:
    - Scalability
    - Fault isolation
    - Independent deployment

© UMHackathon 2026

Email: umhackathon@um.edu.my

Dependency Diagram



2.2.2. Sequence Diagram

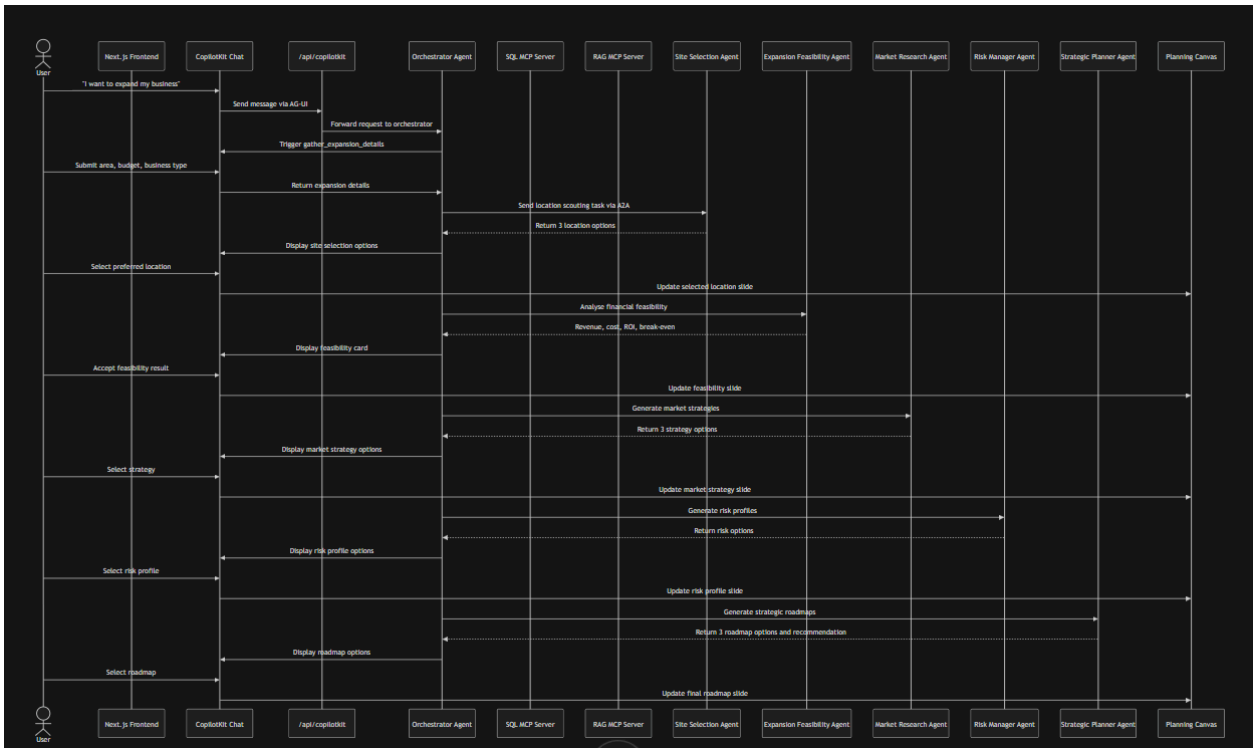
For Inventra, the selected user interaction flow is the AI Business Expansion Planning Flow, where a kiosk owner asks the system to recommend a new business location and generate a complete expansion strategy.

User Journey: AI Business Expansion Planning

1. The kiosk owner opens the Inventra dashboard and uses the AI chat.
2. The user asks: “I want to expand my business.”
3. The frontend sends the request to the CopilotKit API.
4. The Orchestrator Agent detects the intent as Expansion Planning.
5. The system asks the user to provide:
    - Target area
    - Budget range
    - Business type
6. The Site Selection Agent recommends 3 possible locations.

© UMHackathon 2026

7. The user selects one location.
8. The Expansion Feasibility Agent calculates cost, revenue, ROI, and break-even period.
9. The Market Research Agent generates market strategies.
10. The Risk Manager Agent creates risk profiles.
11. The Strategic Planner Agent generates the final expansion roadmap.
12. The selected outputs are displayed on the Planning Canvas.



2.2 Technological Stack

2.2.1 Frontend

The frontend of Inventra is built using Next.js (React framework) with TypeScript and TailwindCSS.

- **Next.js** enables server-side rendering and efficient routing
- **React** provides a dynamic and component-based UI
- **TailwindCSS** ensures responsive and modern styling
- **CopilotKit** integrates AI chat directly into the UI
- **Recharts** is used for data visualization (charts and graphs)
- **Three.js (React Three Fiber)** is used for 3D inventory visualization

This stack allows the system to deliver a real-time, interactive, and user-friendly dashboard.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2.2 Backend

The backend is implemented using a Python-based multi-agent architecture.

- **FastAPI** is used to expose RESTful endpoints for each agent
- **Google ADK (Agent Development Kit)** is used to build LLM-powered agents
- **A2A (Agent-to-Agent Protocol)** enables communication between specialized agents
- **MCP (Model Context Protocol)** allows structured tool access (SQL & RAG)

The backend is responsible for:

- Processing user requests
- Orchestrating AI workflows
- Handling forecasting and decision-making logic

2.2.3 Database

Inventra uses **Supabase (PostgreSQL)** as its primary database.

- Stores:
  - Orders and transactions
  - Inventory stock levels
  - Expiry data
  - Outlet locations
- Supports real-time updates for inventory tracking
- Includes pgvector extension for embedding-based search (RAG)

This ensures **data consistency, scalability, and efficient querying**.

2.2.4 Cloud / Deployment

Inventra adopts a **cloud-based distributed deployment model**:

- **Google Cloud Run** → hosts ML forecasting service
- **Supabase Cloud** → managed PostgreSQL database
- **Localhost / Containerized Services** → runs AI agents (orchestrator + A2A agents)
- **Vercel (or similar)** → frontend deployment (Next.js)

This architecture enables:

- Scalability (independent services)
- High availability
- Easy deployment and maintenance

© UMHackathon 2026

2.3 Key Data Flows

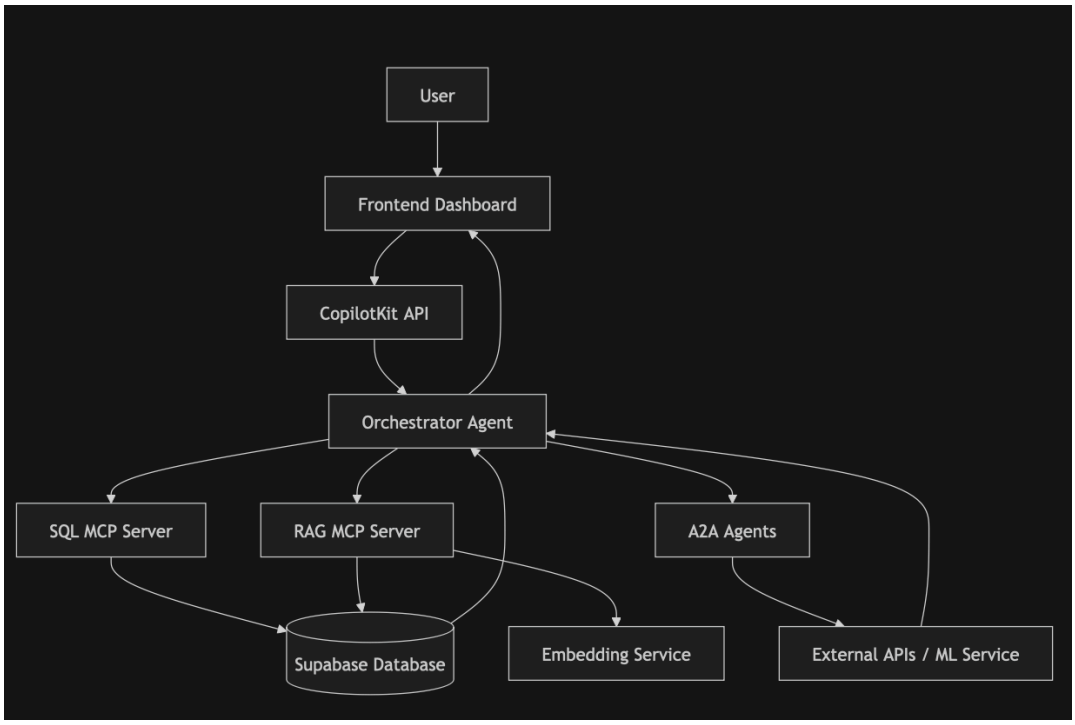
2.3.1 Overview

Inventra processes data through a combination of **user interaction, AI processing, and database operations**.

Data flows across the system in the following stages:

1. User inputs request via frontend (dashboard or chat)
2. Request is sent to backend orchestrator
3. Orchestrator routes request to:
    - SQL MCP (structured data)
    - RAG MCP (contextual knowledge)
    - A2A agents (complex workflows)
4. External APIs and ML services are called when needed
5. Processed data is returned to frontend for visualization

Data Flow Diagram (DFD)



© UMHackathon 2026

Email: umhackathon@um.edu.my

2.3.2 Normalized Database Schema

Inventra's database is designed following Third Normal Form (3NF) to ensure:

- No redundant data
- Proper relationships between entities
- Data integrity and consistency

Key Relationships

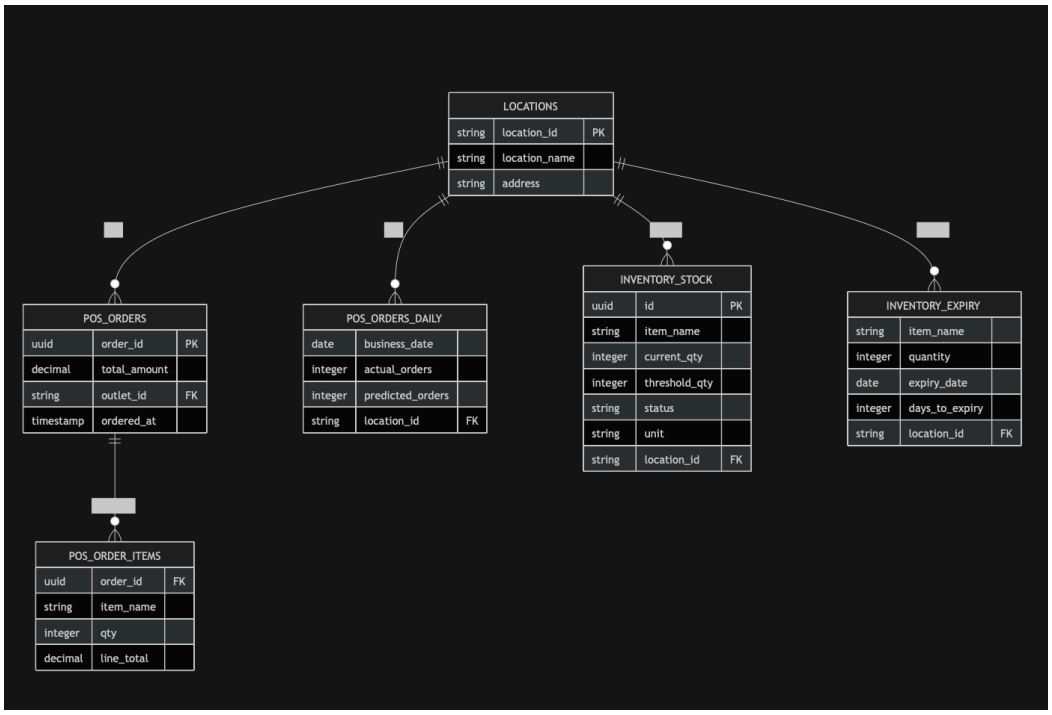
- **Orders** → **Order Items** (1:N relationship)
- **Inventory** → **Expiry Records** (1:N relationship)
- **Locations** → **Orders / Inventory** (via `location_id`)

Design Justification

- Each table stores a single type of entity (no duplication)
- Foreign keys are used to maintain relationships
- Derived data (e.g., totals, predictions) is computed dynamically instead of stored

This design ensures the system remains **scalable, efficient, and maintainable**.

ERD Diagram



© UMHackathon 2026

Email: umhackathon@um.edu.my

Inventra's database schema is designed in a normalized structure to reduce data duplication and maintain data integrity. The `locations` table acts as the central reference table for all outlet-related data. Each outlet can have many orders, daily order summaries, inventory stock records, and expiry records.

The `pos_orders` table stores order-level transaction data, while `pos_order_items` stores item-level details for each order. This separates order information from order item details and supports a one-to-many relationship between orders and order items.

The `inventory_stock` table stores current stock information, while `inventory_expiry` stores expiry-specific records for perishable items. This separation allows the system to track both quantity levels and expiry risks more accurately.

Overall, the schema follows Third Normal Form (3NF) because each table stores one main entity, non-key attributes depend on the primary key, and repeated data is minimized.

3. Functional Requirements & Scope

3.1 Minimum Viable Product:

3.1.1

#	*Features*	*Description*
1	Real-Time Analytics Dashboard	Provides a centralized dashboard displaying key business metrics such as total orders, revenue, average order value, stock levels, and expiry alerts across multiple outlets in real time.
2	AI-Powered Business Expansion Planning	A multi-agent AI system that guides users through selecting a new business location, analyzing financial feasibility, evaluating market strategies, assessing risks, and generating a complete expansion roadmap.
3	ML-Based Demand Forecasting	Predicts future orders, revenue, stockout risk, and inventory requirements using historical data and machine learning models to support proactive decision-making.
4	3D Inventory Management System	Interactive 3D visualization of kiosk inventory that allows users to monitor stock levels, identify critical items, and manage expiry risks with real-time updates.
5	AI Chat-Based Decision Intelligence	Enables users to interact with the system using natural language to retrieve business insights, query performance data, and trigger intelligent workflows such as forecasting and expansion planning.

© UMHackathon 2026

3.2 Non-Functional Requirements (NFRs)

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	The system must handle multiple kiosk outlets and increasing user queries, especially during peak business hours.	Use distributed architecture with independent AI agents (A2A) and scalable cloud services (Supabase, Cloud Run). Each service can scale independently.
Reliability	The system must ensure accurate data retrieval and consistent AI responses with minimal downtime.	Use Supabase managed database with high availability, implement fallback mechanisms (latest available data if real-time data is missing), and error handling in orchestrator.
Maintainability	The system should be modular and easy to update or extend with new features or agents.	Use microservices-based architecture where each AI agent (forecast, risk, market, etc.) is independently developed and deployed.
Latency	AI responses and system queries must return within 3–5 seconds under normal load.	Use Gemini 2.5 Flash (fast inference), asynchronous API calls, and efficient data filtering before sending to LLM.
Security	Sensitive data such as API keys and database access must be protected.	Store API keys in environment variables (.env), use Supabase role-based access (anon vs service role), and secure API endpoints.
Usability	The system must provide an intuitive and user-friendly interface for non-technical kiosk owners.	Use clean dashboard UI, visual charts, and natural language AI chat for easy interaction.
Data Consistency	All business data (orders, inventory, expiry) must remain consistent across the system.	Use relational database (PostgreSQL) with foreign keys and normalized schema (3NF).
Cost Efficiency	The system must minimize API usage cost, especially for LLM and external APIs.	Limit context size, use top-k RAG retrieval, cache repeated queries, and reduce unnecessary API calls.
Extensibility	The system should allow integration of additional AI agents or services in the future.	Use A2A architecture where new agents can be added without modifying existing components.

© UMHackathon 2026

3.3 Out of Scope / Future Enhancements

The following features are part of Inventra's long-term vision but are not included in the current UMHackathon 2026 implementation due to time and scope constraints.

a. Mobile Application (iOS & Android)

A dedicated mobile application for kiosk owners to monitor operations, receive alerts, and interact with the AI system on the go.

b. IoT-Based Smart Inventory Integration

Integration with IoT sensors (e.g., weight sensors, smart shelves) to automatically track inventory levels in real time without manual input.

c. Advanced AI Recommendation Engine

Personalized recommendations based on historical performance, such as:

- Best-selling menu optimization
- Dynamic pricing strategies
- Staff scheduling optimization

d. Automated Supplier & Reordering System

An intelligent procurement system that:

- Automatically generates purchase orders
- Connects with suppliers
- Optimizes reorder timing and quantity

e. Multi-User Collaboration & Role Management

Support for multiple users (owner, manager, staff) with role-based access control and collaborative decision-making features.

f. Real-Time POS System Integration

Direct integration with external POS systems to enable:

- Live transaction syncing
- Unified data pipeline
- Reduced manual data entry

© UMHackathon 2026

g. Predictive Financial Planning & Simulation

Advanced financial simulation tools allowing users to:

- Test different expansion scenarios
- Simulate risks and ROI under different market conditions

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation:

4.1.1 Grayscale Rollout & A/B Testing

Inventra adopts a grayscale rollout strategy to ensure system stability when introducing new features, especially AI-driven functionalities.

- New features (e.g., AI expansion planning or forecasting improvements) are first released to a small subset of users (5–10%)
- System performance, response accuracy, and error rates are monitored
- If no critical issues are detected, the rollout is gradually expanded to **100% of users**

For optimization, **A/B testing** is applied:

- Different AI strategies or model configurations are tested (e.g., different forecasting approaches or recommendation logic)
- Performance metrics such as:
  - Prediction accuracy
  - User engagement
  - Response time
- The better-performing variant is selected for full deployment

This ensures continuous improvement while minimizing risk.

4.1.2 Emergency Rollback (ER) & Golden Release

To ensure system reliability, Inventra implements an **Emergency Rollback (ER) strategy** with a predefined **Golden Release version**.

- A stable version of the system (Golden Release) is always maintained
- If anomalies are detected (e.g., API failure, incorrect AI output, system crash):
  - The system automatically triggers rollback
  - Traffic is redirected to the stable version

© UMHackathon 2026

Trigger Conditions for Rollback

- AI response failure rate exceeds threshold (e.g., >5%)
- External API failure (Google Maps, ML service)
- Database query failure or timeout
- System latency exceeds acceptable limits (>5 seconds)

Recovery Strategy

- Switch to fallback logic (cached data or previous results)
- Disable affected module (e.g., forecasting agent)
- Restore service using Golden Release version

4.1.3 Priority Matrix:

4.1.3.1 Priority Levels and Actions

Priority Level	Trigger Condition	Action Taken
P1 – Critical	- AI system fails to respond- Database unavailable- External API failure (Google Maps / ML Service)- System completely unusable	Immediate **Emergency Rollback** to Golden Release, disable affected modules, switch to fallback (cached or last available data)
P2 – High	- AI response incorrect or inconsistent- Forecast accuracy drops significantly- High latency (>5 seconds)	Restart affected service, retry API calls, fallback to simplified logic, notify development team
P3 – Medium	- Minor UI errors- Partial data missing- Non-critical API delays	Log issue, display fallback message, continue system operation with limited functionality
P4 – Low	- Cosmetic UI issues- Small data inconsistencies- Non-impactful bugs	Fix in next update cycle, no immediate action required

4.2 Monitoring:

4.2.1 System Monitoring Plan

Inventra implements a monitoring strategy to ensure system health, performance, and reliability across all components.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Key Metrics Monitored

- **API Response Time** (target: < 5 seconds)
- **AI Response Success Rate**
- **Database Query Performance**
- **External API Availability**
- **Forecast Accuracy (ML performance)**

Monitoring Mechanism

- Continuous tracking of:
  - Orchestrator response time
  - Agent execution success/failure
- Logging of:
  - Errors and failed API calls
  - AI response anomalies
- Real-time alerts triggered when thresholds are exceeded

Trigger Actions

- High latency → retry or fallback response
- API failure → switch to cached or previous data
- AI failure → disable agent and return safe response
- System overload → limit requests temporarily

4.3 Assumptions

The following assumptions are made during the development and deployment of Inventra:

a. Users (kiosk owners and managers) have **stable internet connectivity** to access the web-based dashboard
b. Kiosk operators have **basic digital literacy** to interact with dashboards and AI chat systems
c. External APIs (Google Maps, Foursquare, ML Service) are **available and responsive**
d. Supabase database service maintains **high availability and consistency**
e. Historical business data is **sufficient for generating accurate forecasts**
f. AI models (Gemini and ML service) produce **reliable and interpretable outputs**

© UMHackathon 2026

4.4 External Dependencies:

Tools	Purpose	Risks
Google Gemini API (LLM)	Core AI reasoning engine for orchestrator and all agents. Handles natural language understanding, multi-step reasoning, and structured output generation.	**High** – If API fails or rate-limited, core AI features (chat, planning, forecasting) will be affected. Mitigation: fallback responses, caching, simplified logic.
OpenAI Embedding API	Generates embeddings for RAG (semantic search) to provide contextual business insights.	**Medium** – If unavailable, system cannot perform semantic search. Mitigation: fallback to direct SQL queries without RAG.
Supabase (PostgreSQL + Realtime)	Primary database for storing orders, inventory, expiry data, and vector embeddings.	**High** – Database downtime affects entire system. Mitigation: use managed cloud service with high availability and backup.
Google Maps API	Provides geolocation, distance calculation, and competitor analysis for site selection.	**Medium** – If unavailable, location analysis becomes less accurate. Mitigation: fallback to static or cached location data.
Foursquare Places API	Used to estimate foot traffic and venue density for location scoring.	**Medium** – Missing data may affect accuracy of site recommendations. Mitigation: use heuristic estimation if API fails.
Bank Negara Malaysia API	Provides financial indicators (e.g., OPR rate) for feasibility analysis.	**Low** – If unavailable, default values can be used.
Malaysia Open Data API	Provides demographic data for market research and strategy generation.	**Low** – Missing data has minimal impact on core functionality.
Google Cloud Run (ML Service)	Hosts ML models for demand forecasting (orders, stockout, revenue prediction).	**High** – If unavailable, forecasting features fail. Mitigation: fallback to historical averages or cached predictions.

© UMHackathon 2026

CopilotKit	Enables AI chat interface and communication between frontend and orchestrator.	**Medium** – If it fails, AI chat functionality is unavailable.
Vercel (Frontend Hosting)	Hosts Next.js frontend application.	**Low** – Downtime affects UI access but not backend logic.

5. Project Management & Team Contributions

5.1 Project Timeline

Day	Phase	Description
Day 1–2	Planning & System Design	Defined problem statement, designed system architecture, selected tech stack, and outlined core features
Day 3–4	Backend Development	Developed orchestrator agent, SQL MCP server, and RAG MCP server
Day 5–6	AI Agent Development	Implemented A2A agents (site selection, feasibility, market, risk, strategy, forecast)
Day 7–8	Frontend Development	Built dashboard, AI chat interface, planning canvas, and inventory UI
Day 9	Integration & Testing	Integrated frontend with backend, tested workflows, fixed bugs, and validated AI responses
Day 10	Final Optimization & Demo Preparation	Performance tuning, UI improvements, and preparation of presentation/demo

5.2 Team Members Role:

1. Fahmi - Product & Business Lead

- Handles PRD, business model, pitch narrative, presentation strategy

2. Izzul - Operations & Strategy Lead

- Handles SAD coordination, documentation flow, timeline, deck organization

Technical side

1. Adli - Technical Lead / CTO

- Handles system architecture, technical direction, integration decisions

2. Zaid - AI / Backend Engineer

- Handles GLM integration, backend logic, API flow, model behavior

© UMHackathon 2026

3. Haziq - Frontend / Prototype Engineer
- Handles UI, prototype, demo flow, usability

5.3 Recommendations

The following recommendations are proposed to improve the system in terms of scalability, performance, and reliability:

- **Implement Redis Caching**
  - Cache frequently requested data (e.g., dashboard queries, AI responses) to reduce latency and API cost
- **Containerization (Docker)**
  - Deploy each agent as a container for better scalability and deployment consistency
- **Load Balancing & Scaling**
  - Use cloud load balancers and auto-scaling for handling high traffic
- **Advanced Monitoring Tools**
  - Integrate monitoring platforms (e.g., Prometheus, Grafana) for real-time system health tracking
- **Model Optimization**
  - Fine-tune ML models with more data to improve forecasting accuracy

© UMHackathon 2026